COA Practical Viva Questions

Experiments 2 to 10 - from very basic to advanced

Every question comes from your practicals. Each section starts with **Basic** questions (nearly every examiner asks these), then **Intermediate**, then **Advanced** (for top marks). Learn all Basic and Intermediate answers first, then read the Advanced ones if time is left.

Section	Basic	Intermediate	Advanced	Total
Common 8086 / DOS questions (asked in every practical)	8	4	1	13
Experiment 2 - ALU operations (ADD, SUB, MUL)	5	6	4	15
Experiment 3 - HEX to BCD conversion	6	6	5	17
Experiment 4 - String operations (reverse and display)	6	8	6	20
Experiment 5 - BCD addition with keyboard input (INT 21H)	5	7	4	16
Experiment 6 - Booth's multiplication algorithm	5	6	7	18
Experiment 7 - Restoring division algorithm	5	7	5	17
Experiment 8 - Direct mapping of cache memory	6	8	9	23
Experiment 9 - Drawing shapes with BIOS INT 10H	7	8	7	22
Experiment 10 - File operations using DOS interrupts in C	6	10	8	24
All sections	**59**	**70**	**56**	**185**

Tip: if you are asked something that is not here, answer from the aim and theory of the experiment, and always explain *why* you used a register or instruction.

Common 8086 / DOS questions (asked in every practical)

Warm-up questions that apply to all the assembly experiments (2, 3, 4, 5, 9).

Basic level

Q1. What is an interrupt?

Ans: A signal that makes the CPU stop what it is doing, run a small service routine (interrupt service routine) and then return. In 8086 a software interrupt is called with the `INT n` instruction.

Q2. What is INT 21H?

Ans: The main DOS service interrupt. The function number is put in AH before calling it, e.g. 01H read key, 02H print char, 09H print string, 4CH terminate, 3CH to 40H file services.

Q3. What is INT 10H?

Ans: The BIOS video interrupt: sets video mode, plots pixels, moves the cursor, writes characters.

Q4. What is the difference between BIOS and DOS interrupts?

Ans: BIOS interrupts (INT 10H video, INT 16H keyboard) are in ROM and talk to hardware directly. DOS interrupts (INT 21H) are provided by the operating system and are built on top of BIOS.

Q5. What is the difference between an assembler directive and an instruction?

Ans: A directive (`.MODEL, .DATA, DB, END`) tells the assembler what to do and produces no machine code. An instruction (`MOV, ADD`) is converted to machine code and executed by the CPU.

Q6. What do .MODEL SMALL, .STACK, .DATA and .CODE mean?

Ans: SMALL = one code segment and one data segment. .STACK reserves stack memory. .DATA starts the data segment (variables). .CODE starts the program instructions.

Q7. Why do we write MOV AX,@DATA and MOV DS,AX?

Ans: To make DS point to our data segment. The 8086 has no instruction to move an immediate value straight into a segment register, so we go through AX.

Q8. How does a program end and return to DOS?

Ans: `MOV AH,4CH` then `INT 21H`. It terminates cleanly and returns control to DOS.

Intermediate level

Q9. How is a physical address formed in the 8086?

Ans: Physical address = segment x 16 + offset (segment shifted left by 4 bits, then added to the offset). This gives a 20-bit address, so 1 MB of memory.

Q10. What are the general purpose registers of the 8086?

Ans: AX, BX, CX, DX (16-bit, each splits into high and low bytes such as AH and AL). AX = accumulator, BX = base, CX = counter (LOOP, REP), DX = data (I/O, MUL and DIV extension).

Q11. What is the Interrupt Vector Table?

Ans: A table of 256 entries x 4 bytes (IP and CS) at address 0000:0000, 1 KB in total. The address of the handler for INT n is stored at n x 4. So INT 21H is at 0000:0084H.

Q12. What is the difference between MASM and DEBUG?

Ans: MASM is the assembler (with LINK) that converts a .ASM source to an .EXE. DEBUG is a small tool to load a program, trace it step by step (T), view registers (R) and memory (D) and run it (G).

Advanced level

Q13. What do the flags do? Name the important ones.

Ans: Flags hold the status of the last operation: CF carry, ZF zero, SF sign, OF overflow, PF parity, AF auxiliary carry; and control flags DF (direction) and IF (interrupt enable), TF (trap/single-step).

Experiment 2 - ALU operations (ADD, SUB, MUL)

Addition, subtraction and multiplication in 8086 assembly.

Basic level

Q14. What is an ALU?

Ans: The Arithmetic Logic Unit of the CPU. It does arithmetic (add, subtract, multiply, divide) and logic (AND, OR, NOT, XOR) operations.

Q15. What does `ADD AL, NUM2` do?

Ans: It adds the byte at NUM2 to AL and stores the result back in AL (AL = AL + NUM2). 05H + 03H = 08H.

Q16. What does `SUB AL, NUM2` do?

Ans: AL = AL - NUM2. 05H - 03H = 02H.

Q17. What do DB and DW mean?

Ans: DB defines a byte (8 bits), DW defines a word (16 bits).

Q18. Which register holds the result of MUL BL?

Ans: AX. For an 8-bit multiply the result is 16 bits, AH = high byte and AL = low byte.

Intermediate level

Q19. What is multiplied when we write `MUL BL`?

Ans: AL x BL. AL is implied, we do not write it.

Q20. What happens for a 16-bit multiplication, e.g. `MUL BX`?

Ans: AX x BX gives a 32-bit result stored in DX:AX (DX = high word, AX = low word).

Q21. What is the difference between MUL and IMUL?

Ans: MUL is for unsigned numbers, IMUL for signed numbers (2's complement).

Q22. Which flags are affected by ADD and SUB?

Ans: CF, ZF, SF, OF, PF, AF.

Q23. What happens to the result if FFH + 01H is executed on AL?

Ans: AL becomes 00H and the Carry Flag is set to 1 (the 9th bit is lost into CF), and ZF = 1.

Q24. How do you verify the result in DEBUG?

Ans: Load with `DEBUG EXP2.EXE`, use T to trace instruction by instruction, R to see registers (AL = 08 after ADD, AX = 000F after MUL) and G to run, Q to quit.

Advanced level

Q25. What is the difference between SUB and CMP?

Ans: Both subtract and set flags, but CMP does not store the result; it only changes the flags (used before conditional jumps).

Q26. What is the largest 8-bit unsigned product, and what do CF and OF show after MUL?

Ans: FFH x FFH = FE01H. For MUL, CF and OF are set to 1 when the upper half (AH) is not zero, meaning the result does not fit in 8 bits.

Q27. How would you add two 32-bit numbers on the 8086?

Ans: Add the lower words with ADD, then the upper words with ADC (add with carry) so the carry from the first addition is included.

Q28. Why can't you write `ADD AL, NUM` if NUM is defined with DW?

Ans: Operand sizes must match: AL is 8-bit and NUM is 16-bit, so the assembler gives a size mismatch error. Use AX, or `BYTE PTR NUM`.

Experiment 3 - HEX to BCD conversion

Convert a hex number to decimal (BCD) digits and print them.

Basic level

Q29. What is a hexadecimal number?

Ans: A base-16 number system using digits 0-9 and A-F (A = 10 ... F = 15). One hex digit = 4 bits.

Q30. What is BCD?

Ans: Binary Coded Decimal: each decimal digit is stored separately in 4 bits. Decimal 37 is 0011 0111 in BCD.

Q31. What is the difference between BCD and ordinary binary?

Ans: In binary the whole number is converted (37 = 100101). In BCD every digit is coded separately (37 = 0011 0111).

Q32. What is packed and unpacked BCD?

Ans: Packed: two digits per byte (37 = 37H). Unpacked: one digit per byte with the upper nibble 0 (03H, 07H).

Q33. What is 25H in decimal?

Ans: 2 x 16 + 5 = 37.

Q34. Why do we add 30H before printing a digit?

Ans: INT 21H prints ASCII characters. The ASCII code of '0' is 30H, so digit + 30H gives the code of that digit character.

Intermediate level

Q35. Why do we divide by 10?

Ans: Dividing a number by 10 gives the tens digit as quotient and the ones digit as remainder.

Q36. Where are the quotient and remainder after `DIV BL`?

Ans: Quotient in AL, remainder in AH (for an 8-bit divisor the dividend is AX).

Q37. Why do we clear AH (`MOV AH,00H`) before DIV BL?

Ans: The dividend is the 16-bit AX. If AH contains garbage, the dividend is wrong and the quotient may not fit in AL.

Q38. Why is the remainder copied to BH?

Ans: Because AH is overwritten by `MOV AH,02H` (the print function number) before the ones digit is printed.

Q39. What does `MOV AH,02H / INT 21H` do?

Ans: Prints the single character whose ASCII code is in DL.

Q40. What is the limitation of the program?

Ans: It divides by 10 only once, so it is correct only up to 99 decimal (63H). For FFH (255) the quotient is 25, which prints a wrong character. A third digit needs a division by 100 first.

Advanced level

Q41. What happens if the quotient does not fit in AL?

Ans: A divide overflow occurs and the 8086 generates INT 0 (divide error).

Q42. How would you get packed BCD instead of printing?

Ans: `MOV CL,4`, `SHL AL,CL` (move tens digit to the upper nibble), then `OR AL,AH`. For 25H you get 37H.

Q43. What is the DAA instruction and is it used here?

Ans: DAA (decimal adjust after addition) corrects AL after adding two packed BCD numbers. It is not used for hex to BCD conversion; here the division method is used.

Q44. What does AAM do?

Ans: AAM divides AL by 10 and puts the quotient in AH and the remainder in AL (unpacked BCD). It is a one-instruction alternative to DIV for numbers below 100.

Q45. How would you convert a 16-bit hex number to BCD?

Ans: Repeatedly divide the number by 10 using 16-bit DIV (DX:AX / 10). Push each remainder on the stack, then pop and print the digits so they come out in the right order.

Experiment 4 - String operations (reverse and display)

Reversing a string with the stack, displaying a string, and 8086 string instructions.

Basic level

Q46. What is a string in assembly language?

Ans: A sequence of bytes (or words) stored in consecutive memory locations.

Q47. Why must the string end with $ for INT 21H, AH=09H?

Ans: Function 09H prints characters until it finds the $ character. Without $ it keeps printing memory contents.

Q48. What is the difference between AH=09H and AH=02H?

Ans: 09H prints a whole string whose address is in DX. 02H prints one character that is in DL.

Q49. What is a stack?

Ans: A LIFO (last in, first out) memory area. PUSH stores a value at the top, POP takes it back.

Q50. How does the program reverse the string?

Ans: It pushes every character on the stack, then pops them one by one. The last pushed comes out first, so the string comes out reversed.

Q51. What is the output for 'Hi C Division'?

Ans: noisiviD C iH

Intermediate level

Q52. Why is CX set to 13?

Ans: CX is the loop counter. 'Hi C Division' has 13 characters (H, i, space, C, space, D, i, v, i, s, i, o, n). With 12 the last letter would be missed.

Q53. What does the LOOP instruction do?

Ans: It decrements CX by 1 and jumps to the label if CX is not zero.

Q54. Why do we push AX (16 bits) and not just the character?

Ans: PUSH and POP work only on 16-bit registers. So the character is put in AL, AH is cleared, and AX is pushed.

Q55. What is the difference between OFFSET and LEA?

Ans: Both give an address. OFFSET is calculated by the assembler at assembly time (`MOV SI,OFFSET STR1`). LEA calculates it at run time (`LEA DX,K1`).

Q56. What are SI and DI?

Ans: SI = source index (string read from DS:SI). DI = destination index (string written to ES:DI).

Q57. What do MOVSB, LODSB, STOSB, SCASB and CMPSB do?

Ans: MOVSB copies a byte DS:SI to ES:DI. LODSB loads DS:SI into AL. STOSB stores AL into ES:DI. SCASB compares AL with ES:DI. CMPSB compares DS:SI with ES:DI. All of them update SI/DI automatically.

Q58. What is the direction flag? What are CLD and STD?

Ans: DF decides whether SI and DI increase (DF = 0, CLD) or decrease (DF = 1, STD) after each string instruction.

Q59. What does the REP prefix do?

Ans: It repeats the string instruction CX times (decrementing CX each time). Example: `REP MOVSB` copies CX bytes.

Advanced level

Q60. What is the difference between REP, REPE/REPZ and REPNE/REPNZ?

Ans: REP repeats while CX is not 0. REPE/REPZ repeats while CX is not 0 and ZF = 1 (strings equal so far). REPNE/REPNZ repeats while CX is not 0 and ZF = 0 (not found yet).

Q61. Why must ES be loaded for MOVSB?

Ans: The destination is always ES:DI. In a small-model program set `MOV ES,AX` the same as DS.

Q62. Are MOVSD, LODSD and others available on the 8086?

Ans: No. The 32-bit (double word) string instructions are available from the 80386 onwards. The 8086 has only byte (B) and word (W) versions.

Q63. How can you check a palindrome using the reverse program?

Ans: Reverse the string using the stack, then compare each popped character with the original from the start (using CMP, or REPE CMPSB). If all match it is a palindrome.

Q64. How do you find the length of a string using SCASB?

Ans: Set AL to the terminator, CX = 0FFFFH, then `REPNE SCASB`, then `NOT CX` and `DEC CX`. CX holds the length without the terminator.

Q65. What bugs existed in the lab code?

Ans: CX was 12 instead of 13, the program had no exit (`MOV AH,4CH / INT 21H`), and in the display program `RET` was written after `END MAIN` where it is ignored.

Experiment 5 - BCD addition with keyboard input (INT 21H)

Read two digits, add them and print the 2-digit result.

Basic level

Q66. Which function reads a character from the keyboard?

Ans: INT 21H, AH = 01H. The ASCII code of the key comes back in AL (and it is echoed on the screen).

Q67. Why do we subtract 30H from the typed key?

Ans: The keyboard gives ASCII. '5' is 35H. Subtracting 30H gives the number 5.

Q68. What do 13 and 10 in the message definition mean?

Ans: 13 = carriage return (CR) and 10 = line feed (LF). Together they move the cursor to the start of the next line.

Q69. What does AH = 09H do?

Ans: Displays the $-terminated string whose address is in DX.

Q70. What is the maximum result of adding two BCD digits?

Ans: 9 + 9 = 18, so two digits are needed for the output.

Intermediate level

Q71. How do we get the two digits of the result?

Ans: Divide the sum by 10 (`DIV BL` with BL = 10). Quotient in AL = tens digit, remainder in AH = ones digit. Add 30H to each and print.

Q72. Why was the sum saved in CL before printing 'Result ='?

Ans: INT 21H/09H returns with AL = 24H (the '$'), which would destroy the sum kept in AL. Saving it in CL and restoring it avoids this.

Q73. Why is `MOV AH,00H` needed before DIV BL?

Ans: The dividend for DIV BL is AX. AH must be 0 or the dividend becomes wrong.

Q74. What is the difference between AH = 01H and AH = 08H?

Ans: Both read a key into AL. 01H echoes the key on the screen, 08H does not echo.

Q75. What does the DAA instruction do?

Ans: Decimal Adjust after Addition. After adding two packed BCD numbers it corrects AL: if the low nibble is greater than 9 (or AF = 1) add 06H; if the high nibble is greater than 9 (or CF = 1) add 60H.

Q76. Give a DAA example.

Ans: 09H + 09H = 12H in binary (AF is set because of a carry from bit 3). DAA adds 06H and gives 18H, which is the correct BCD for 18.

Q77. What is the difference between DAA and AAA?

Ans: DAA works on packed BCD (two digits per byte). AAA works on unpacked BCD (one digit per byte).

Advanced level

Q78. How would you add two 2-digit BCD numbers?

Ans: Read the digits and pack them into bytes (tens x 16 + ones), `ADD AL,BL`, then `DAA`. Use ADC to catch a carry into the hundreds digit, then print the nibbles.

Q79. How would you reject input like the letter 'A'?

Ans: Compare the key with 30H and 39H: `CMP AL,'0' / JB invalid` and `CMP AL,'9' / JA invalid`, then ask again.

Q80. What happens if the sum is more than 99 in a packed BCD addition?

Ans: Carry flag is set. Use `ADC AH,0` to collect the carry as the hundreds digit.

Q81. Why does the program print 09 and not 9 for 4 + 5?

Ans: It always prints a tens digit. To suppress the leading zero, compare the tens digit with 0 and skip printing it.

Experiment 6 - Booth's multiplication algorithm

Signed multiplication using 2's complement and Booth recoding.

Basic level

Q82. What is Booth's algorithm?

Ans: A method to multiply two signed binary numbers in 2's complement form using add, subtract and shift operations.

Q83. Why do we need Booth's algorithm?

Ans: Ordinary shift-and-add works for unsigned numbers and needs special handling for negative numbers. Booth's gives one uniform method for positive and negative numbers and needs fewer additions when the multiplier has runs of 1s or 0s.

Q84. What is 2's complement?

Ans: The way signed numbers are stored: invert all bits and add 1. For example -3 in 4 bits: 0011 becomes 1100, plus 1 gives 1101.

Q85. Name the registers used.

Ans: M = multiplicand, Q = multiplier, A = accumulator (starts 0), Q-1 = extra bit (starts 0), Count = number of bits n.

Q86. State Booth's rule.

Ans: Look at Q0 and Q-1. 10 means A = A - M. 01 means A = A + M. 00 or 11 means no operation. Then arithmetic shift right (A, Q, Q-1).

Intermediate level

Q87. How many iterations are there?

Ans: n, the number of bits of the multiplier. (8 for 8-bit numbers.)

Q88. What is the size of the product and where is it?

Ans: 2n bits, stored in A and Q joined together (A is the upper half, Q the lower half).

Q89. What is an arithmetic shift right?

Ans: Shift right where the sign bit (MSB) is kept and copied, so the sign of a negative number is not lost. Here A's LSB goes into Q's MSB and Q0 goes into Q-1.

Q90. What is the role of Q-1 and why is it 0 initially?

Ans: It stores the bit shifted out of Q0, so we can compare each bit with its right neighbour. Starting with 0 treats the multiplier as having a 0 on the right of its LSB.

Q91. Trace 3 x (-2) with 4 bits.

Ans: M = 0011, Q = 1110. Step 1: 00, shift: A=0000 Q=0111. Step 2: 10, A = A - M = 1101, shift: A=1110 Q=1011. Step 3: 11 shift: A=1111 Q=0101. Step 4: 11 shift: A=1111 Q=1010. Product = 11111010 = -6.

Q92. What does a pair 01 or 10 mean in terms of the multiplier bits?

Ans: 10 = the start (from the right) of a run of 1s, 01 = the end of a run of 1s. 00 and 11 = middle of a run of 0s or 1s.

Advanced level

Q93. Why does subtracting at the start and adding after the end of a run of 1s work?

Ans: A run of 1s from bit i to bit j equals 2^(j+1) - 2^i. So instead of adding each 1, subtract M x 2^i once and add M x 2^(j+1) once.

Q94. How many add/subtract operations are needed for the multiplier 0111 1110?

Ans: Only 2 (a subtract at the start of the run and an add after its end), instead of 6 additions.

Q95. What is the worst case for Booth's algorithm?

Ans: A multiplier with alternating bits like 0101 0101: every step needs an add or subtract (n operations).

Q96. What if the multiplier is 0?

Ans: All pairs are 00, so only shifts happen and the product is 0.

Q97. What is modified Booth (radix-4) recoding?

Ans: It examines 3 bits at a time (overlapping) and so produces only n/2 partial products, making hardware multipliers about twice as fast.

Q98. Why must the shift be arithmetic and not logical?

Ans: A logical shift would put 0 into the sign position and a negative partial product would turn positive, giving a wrong answer.

Q99. What is the problem when the multiplicand is the most negative number, e.g. -128 in 8 bits?

Ans: Its negative (+128) cannot be represented in 8 bits, so hardware needs an extra bit for A. In the C program A is a full int, so it works.

Experiment 7 - Restoring division algorithm

Unsigned division with shift, subtract and restore.

Basic level

Q100. What is the restoring division algorithm?

Ans: A division method that mimics long division with shift, subtract and check sign. If the subtraction goes negative, the divisor is added back (restored).

Q101. Why is it called 'restoring'?

Ans: Because when A - M is negative, the old value of A is restored by adding M back.

Q102. Name the registers and their initial values.

Ans: A = 0 (partial remainder), Q = dividend, M = divisor, n = number of bits (counter).

Q103. Where are the quotient and remainder at the end?

Ans: Quotient in Q, remainder in A.

Q104. Write the steps of the algorithm.

Ans: 1) A = 0, Q = dividend, M = divisor, count = n. 2) Shift (A,Q) left. 3) A = A - M. 4) If A is negative: Q0 = 0 and A = A + M (restore); else Q0 = 1. 5) count = count - 1. 6) Repeat from step 2 until count = 0.

Intermediate level

Q105. How many iterations are needed?

Ans: n iterations for an n-bit dividend.

Q106. How do you know the subtraction was wrong?

Ans: The sign bit (MSB) of A becomes 1, i.e. A is negative, meaning M was larger than the current partial remainder.

Q107. What does the shift of (A,Q) left do?

Ans: It moves the next bit of the dividend into A, like 'bringing down' the next digit in long division. The empty LSB of Q is where the new quotient bit goes.

Q108. Trace 11 / 3 (4 bits).

Ans: Step 1: A=00001, A-M negative so Q=0110 restore. Step 2: A=00010, negative, Q=1100. Step 3: A=00101-00011 = 00010, Q=1001. Step 4: A=00101-00011 = 00010, Q=0011. Quotient 3, remainder 2.

Q109. How can you verify the answer?

Ans: Dividend = Divisor x Quotient + Remainder. 3 x 3 + 2 = 11.

Q110. Why does A need n + 1 bits?

Ans: To hold the sign bit when the subtraction gives a negative value.

Q111. What must the program check before dividing?

Ans: That the divisor is not 0 (division by zero is undefined).

Advanced level

Q112. What is the difference between restoring and non-restoring division?

Ans: Non-restoring does not restore. If A is negative it shifts and ADDS M in the next step; if A is positive it shifts and SUBTRACTS M. A single correction (add M if the final A is negative) is done at the end. It needs only one add/subtract per step.

Q113. What is the worst-case number of arithmetic operations in restoring division?

Ans: 2n: every step can need a subtraction and a restoring addition.

Q114. Which is faster, restoring or non-restoring and why?

Ans: Non-restoring, because it removes the extra restore addition in each step.

Q115. How would you divide signed numbers with this algorithm?

Ans: Divide the absolute values, then give the quotient a negative sign if the signs of dividend and divisor differ; the remainder takes the sign of the dividend.

Q116. How does 8086 perform division in hardware?

Ans: Using the DIV (unsigned) and IDIV (signed) instructions. DIV BL gives AL = quotient and AH = remainder; DIV BX divides DX:AX and gives AX = quotient, DX = remainder.

Experiment 8 - Direct mapping of cache memory

Cache basics and the direct mapping formula.

Basic level

Q117. What is cache memory?

Ans: A small, very fast memory placed between the CPU and main memory that holds recently used data and instructions.

Q118. Why do we need cache memory?

Ans: The CPU is much faster than main memory. Cache reduces the waiting time and improves performance.

Q119. What are cache hit and cache miss?

Ans: Hit: the required data is found in the cache. Miss: it is not in the cache and must be fetched from main memory.

Q120. What is the hit ratio?

Ans: Hit ratio = number of hits / total number of memory accesses.

Q121. Name the three cache mapping techniques.

Ans: Direct mapping, associative mapping and set-associative mapping.

Q122. State the formula of direct mapping.

Ans: Cache line number = (Main memory block number) MOD (Number of cache lines).

Intermediate level

Q123. Calculate the cache line for address 25 with 8 lines.

Ans: 25 MOD 8 = 1, so cache line 1.

Q124. Into which parts is the address divided in direct mapping?

Ans: Tag | Line (index) | Byte offset (word offset).

Q125. What is the use of the tag?

Ans: Many memory blocks map to the same line. The tag stored with the line tells which block is currently in it; the cache compares it with the tag of the address to decide hit or miss.

Q126. What happens when two blocks map to the same line?

Ans: The new block replaces the old one (conflict). The old block must be fetched again if needed.

Q127. What is the advantage and disadvantage of direct mapping?

Ans: Advantage: simple, cheap and fast (only one tag comparison, no replacement algorithm). Disadvantage: conflict misses and a lower hit ratio when blocks that map to the same line are used alternately.

Q128. Is a replacement policy needed in direct mapping?

Ans: No. Each block has only one possible line, so there is nothing to choose.

Q129. What is associative mapping?

Ans: A block can be placed in any cache line. All tags are compared at once. It is flexible but costly hardware.

Q130. What is set-associative mapping?

Ans: The cache is divided into sets and a block maps to one set but can use any line (way) in that set. It is a compromise between direct and fully associative.

Advanced level

Q131. Find tag, line and offset for address 100, block size 4 words, 8 lines.

Ans: Block number = 100 / 4 = 25, offset = 100 MOD 4 = 0, line = 25 MOD 8 = 1, tag = 25 / 8 = 3.

Q132. A 16-bit address, 8 cache lines, block size 4 words. How many bits are tag, line and offset?

Ans: Offset = 2 bits (4 words), line = 3 bits (8 lines), tag = 16 - 3 - 2 = 11 bits.

Q133. Give an example of thrashing in direct mapping.

Ans: With 8 lines, blocks 1 and 9 both go to line 1. If the program accesses 1, 9, 1, 9 ... every access is a miss even though the other lines are empty.

Q134. Why is the number of lines usually a power of 2?

Ans: Then MOD is just taking the lower bits of the block number, so no divider hardware is needed.

Q135. What are the types of cache misses?

Ans: Compulsory (first access), capacity (cache too small) and conflict (blocks fight for the same line or set).

Q136. What are temporal and spatial locality?

Ans: Temporal: recently used data will be used again soon. Spatial: data near the one just used will be used soon. Cache works because programs show both.

Q137. What are write-through and write-back?

Ans: Write-through updates cache and main memory together. Write-back updates only the cache and writes to memory later when the line is replaced (needs a dirty bit).

Q138. What is average memory access time?

Ans: AMAT = hit time + miss rate x miss penalty.

Q139. What is the limitation of the program in the manual?

Ans: It uses the address directly as block number (block size = 1 word), and it only shows the line number; it does not store data, tags or count hits.

Experiment 9 - Drawing shapes with BIOS INT 10H

Graphics mode 13H, plotting pixels, loops and jumps.

Basic level

Q140. What is the BIOS?

Ans: Basic Input/Output System: firmware in ROM that provides basic services for the keyboard, display, disk and other hardware.

Q141. Which interrupt is used for video services?

Ans: INT 10H.

Q142. Which function plots a pixel?

Ans: AH = 0CH (write graphics pixel).

Q143. What do AL, BH, CX and DX hold for the pixel function?

Ans: AL = colour, BH = display page (0), CX = X coordinate (column), DX = Y coordinate (row).

Q144. What is mode 13H?

Ans: A VGA graphics mode with 320 x 200 pixels and 256 colours.

Q145. How do you switch to graphics mode and come back?

Ans: `MOV AX,0013H / INT 10H` to enter mode 13H and `MOV AX,0003H / INT 10H` to return to 80x25 text mode.

Q146. What is a pixel?

Ans: The smallest dot on the screen that can be given a colour.

Intermediate level

Q147. How do you draw a horizontal line? A vertical line?

Ans: Horizontal: keep DX constant and increase CX in a loop. Vertical: keep CX constant and increase DX.

Q148. What are the coordinate limits in mode 13H and where is the origin?

Ans: X: 0-319, Y: 0-199. The origin (0,0) is the top-left corner and Y increases downwards.

Q149. Why do we use INT 16H, AH = 00H in the program?

Ans: It waits for a key press so the drawing stays on the screen until the user presses a key.

Q150. Why is `JBE` used after `CMP CX,240`?

Ans: JBE (jump if below or equal, unsigned) repeats the loop while CX is 240 or less, so the pixel at 240 is also drawn.

Q151. What is the difference between JBE and JLE?

Ans: JBE is for unsigned comparison (uses CF and ZF). JLE is for signed comparison (uses SF, OF and ZF).

Q152. Which colour is AL = 04H?

Ans: Red in the standard VGA palette.

Q153. How much video memory does mode 13H use?

Ans: 320 x 200 = 64,000 bytes, one byte per pixel (an index into the 256-colour palette), at segment A000H.

Q154. Why is AH and AL loaded again inside each loop?

Ans: BIOS routines may change AX, so the function number and colour are set before every call.

Advanced level

Q155. Why is the apex of the triangle at (160,70)?

Ans: Both sides move 1 pixel across for each 1 pixel up (45 degrees). The base is 160 wide (80 to 240), so the apex is 80 pixels above the base: x = 160, y = 150 - 80 = 70.

Q156. What was wrong with the manual's triangle?

Ans: The left side ran to (180,50) while the right side started at (160,50) and ended at (260,150) but the base stopped at x = 240, so the shape was not closed. Also BH was never set to 0.

Q157. How do you draw a diagonal line?

Ans: Change both coordinates in each step, e.g. `INC CX / DEC DX` for a line going up to the right.

Q158. How would you fill the rectangle?

Ans: Use two nested loops: the outer loop changes DX (rows) and the inner loop plots from CX = 80 to 240 for each row.

Q159. How can you plot pixels without INT 10H?

Ans: Write directly to video memory: set ES = A000H and store the colour byte at offset (Y x 320 + X). This is faster than a BIOS call per pixel.

Q160. Which algorithms draw lines and circles of any slope?

Ans: Bresenham's line algorithm and the midpoint circle algorithm (integer arithmetic only).

Q161. Name other video modes.

Ans: 03H = 80x25 text, 12H = 640x480 with 16 colours, 10H = 640x350 with 16 colours, 13H = 320x200 with 256 colours.

Experiment 10 - File operations using DOS interrupts in C

Create, write, read and close a file with INT 21H from Turbo C.

Basic level

Q162. What is a file handle?

Ans: A small number returned by DOS when a file is created or opened. All later operations (read, write, close) use this number to refer to the file.

Q163. Which interrupt is used for file operations?

Ans: INT 21H, with the function number in AH.

Q164. Give the function numbers for create, open, read, write and close.

Ans: Create = 3CH, open = 3DH, read = 3FH, write = 40H, close = 3EH.

Q165. What is `int86()`?

Ans: A function in dos.h that loads the CPU registers from a REGS structure, executes the given interrupt and returns the registers after the call.

Q166. What is `union REGS`?

Ans: A union that represents the CPU registers: `regs.h.ah` for 8-bit registers, `regs.x.ax` for 16-bit registers and `regs.x.cflag` for the carry flag.

Q167. Why must a file be closed?

Ans: Closing writes remaining data to disk, updates the directory entry (size, date) and frees the handle.

Intermediate level

Q168. How does DOS report an error?

Ans: It sets the Carry Flag (CF = 1) and puts the error code in AX. In C check `regs.x.cflag`.

Q169. What are the access modes of function 3DH?

Ans: AL = 0 read only, 1 write only, 2 read/write.

Q170. What does CX = 0 mean in function 3CH?

Ans: File attribute 'normal'. (1 = read-only, 2 = hidden, 4 = system.)

Q171. What are the inputs of function 40H (write)?

Ans: BX = handle, CX = number of bytes, DS:DX = address of the buffer. It returns the number of bytes written in AX.

Q172. What does function 3FH return at end of file?

Ans: AX = 0 (zero bytes read).

Q173. What happens if 3CH is used on an existing file?

Ans: The file is truncated to zero length, i.e. its old content is lost.

Q174. Why sizeof(buffer) - 1 and `buffer[regs.x.ax] = '\0'`?

Ans: DOS does not add a terminator. AX gives the bytes read, so we place '\0' after them to make a valid C string for printf.

Q175. Why is \r\n used in the data?

Ans: DOS marks the end of a line with carriage return + line feed (CR LF).

Q176. Why is the file name a null-terminated string?

Ans: DOS expects an ASCIZ string (ending with byte 0) in DS:DX.

Q177. Why is the first handle usually 5?

Ans: Handles 0 to 4 are already used by the standard devices: 0 stdin, 1 stdout, 2 stderr, 3 auxiliary, 4 printer.

Advanced level

Q178. What was wrong with the manual's error check?

Ans: It compared the handle with 0xFFFF. DOS errors are indicated by the carry flag, with the error code in AX, so the carry flag must be tested.

Q179. Common DOS error codes?

Ans: 02 file not found, 03 path not found, 04 too many open files, 05 access denied, 06 invalid handle.

Q180. How do you detect a full disk when writing?

Ans: A full disk does not set CF. Compare the bytes written (AX) with the bytes requested (CX); if AX is smaller the disk is full.

Q181. How can you append to a file?

Ans: Open with write access (AL = 1), move the file pointer to the end with function 42H (AL = 2, CX:DX = 0), then write.

Q182. Other useful file functions?

Ans: 41H delete file, 42H move file pointer (LSEEK), 56H rename, 39H make directory, 3AH remove directory, 3BH change directory.

Q183. How is this different from fopen() and fprintf()?

Ans: fopen/fprintf are C library functions that add buffering and formatting and finally call these DOS services. Here we use the low-level service directly.

Q184. Why can we use `(unsigned int)filename` as the DX value?

Ans: In Turbo C's small memory model pointers are 16-bit near pointers, so the offset fits in DX and DS already points to the data segment. In large models you must set the segment explicitly (int86x with segread).

Q185. Why will this program not run directly on modern 64-bit Windows?

Ans: It is a 16-bit DOS program that needs dos.h and DOS interrupts. Run it in DOSBox (or a 32-bit Windows with NTVDM).